

This document describes methods and techniques used and planned for future deployments of the **MedSupplyDrones** project to conduct reviews and improve overall quality. Moreover, the construction of a Continuous Integration pipeline and its benefits are described, along with a demonstration of its implementation.

The Inspection

Review and inspection techniques complement testing by leveraging human expertise to identify issues that automated tests might miss. The following inspection approaches are employed:

- **Code style guidelines and linters:** The project uses Checkstyle for Java code style enforcement, configured to enforce Google Java Style Guide conventions. This ensures consistent formatting, naming conventions, and Javadoc documentation across all modules. The linter runs as part of the Maven build process and fails the build on style violations.
- **Pre-commit checklist:** A checklist of important attributes is verified before each major release: all tests pass locally, coverage thresholds met (>90% line coverage), no compiler warnings, API documentation updated, and performance benchmarks verified against 30-second budget.
- **Self-review and code walkthrough:** Before committing significant changes to DroneRoutingService or GeometryService, I perform a systematic walkthrough of the code against the functional requirements from LO1. This helps identify logical errors and boundary conditions that may not be covered by automated tests.
- **Static analysis with SpotBugs:** SpotBugs (successor to FindBugs) is integrated into the build process to detect potential bugs, including null pointer dereferences, resource leaks, and concurrency issues. This is particularly important for the pathfinding algorithm where subtle bugs could cause incorrect flight paths.
- **CI pipeline notifications:** The CI pipeline sends email notifications when tests fail on pushes to the main branch. This ensures immediate awareness of regression issues, which is critical as the project grows in complexity.

Review Criteria Applied

The following review criteria were applied to selected components:

DroneRoutingService Review

Requirement: Calculate optimal flight paths avoiding no-fly zones within 30-second budget.

Review Focus: Algorithm correctness, boundary handling, performance characteristics.

Issues Identified: (1) Potential infinite loop if A* algorithm encounters unreachable destination – added maximum iteration guard. (2) Floating-point comparison using == instead of tolerance-based comparison – refactored to use 10^{-12} tolerance. (3) Missing null check on polygon vertices from API response – added defensive validation.

GeometryService Review

Requirement: Provide accurate distance calculations and polygon containment checks.

Review Focus: Mathematical correctness, edge cases, numerical stability.

Issues Identified: (1) isCloseTo method used inconsistent threshold (0.00015° vs 0.00014°) – standardised to specification value. (2) Path2D even-odd rule not documented – added explanatory comments. (3) Direction16 enumeration missing validation for invalid angles – added bounds checking.

CI/CD Pipeline

Continuous Integration & Continuous Delivery pipelines are a modern approach combining practices from software engineering to ensure maintainable code quality, enable seamless feature deployment, and eliminate human error. They identify issues early by automating regression testing and informing developers about failing components.

The pipeline includes 4 key stages:

1. Source (Version Control)

Forms the foundation of the pipeline, involving storage and management of the codebase in a versioned manner. This ensures every version and change is tracked, can be retrieved, and can be reverted if something goes wrong. GitHub manages this automatically – developers label commits and use branches for experimental features. The main branch is protected, requiring all CI checks to pass before merging.

2. Build

A critical phase that compiles source code into a deployable artifact. Dependencies are resolved and validated in a remote environment, eliminating "works-on-my-machine" issues. The build uses GitHub Actions workflows specifying JDK 21, Maven for dependency management, and Ubuntu as the build environment. The Spring Boot application is packaged as an executable JAR.

3. Test

The vital stage where unit, integration, and system tests execute to assess application correctness. Testing ensures the application behaves as expected – any failures trigger a notification email to developers. The test stage runs 'mvn test' executing all JUnit 5 tests, followed by 'mvn verify' for integration tests. Coverage reports are generated using JaCoCo and uploaded as build artifacts.

4. Deploy

The final stage where the application is released to the deployment environment. For MedSupplyDrones, the application runs in a Docker container. The Dockerfile builds a lightweight image using Eclipse Temurin JRE 21, exposing port 8080 for the REST API. Deployment could be automated to AWS ECS or Kubernetes, though for the coursework, only local Docker deployment is implemented.

Pipeline Implementation

The pipeline is implemented using GitHub Actions with the following workflow configuration:

```

1  name: MedSupplyDrones CI Pipeline
2
3  on:
4    push:
5      branches: [ "main" ]
6    pull_request:
7      branches: [ "main" ]
8
9  jobs:
10   build:
11     runs-on: ubuntu-latest
12
13     steps:
14       - uses: actions/checkout@v4
15
16       - name: Set up JDK 21
17         uses: actions/setup-java@v4
18         with:
19           java-version: '21'
20           distribution: 'temurin'
21           cache: maven
22
23       - name: Build and Test with Maven
24         run: mvn -B package --file pom.xml

```

The workflow triggers on pushes and pull requests to the main branch. It sets up JDK 21 with Temurin distribution, caches Maven dependencies for faster builds, and runs 'mvn package' which compiles, tests, and packages the application.

build		
succeeded 17 minutes ago in 1m 19s		
Search logs		
> ✓ Set up job		1s
> ✓ Run actions/checkout@v4		0s
> ✓ Set up JDK 21		0s
> ✓ Build and Test with Maven		43s
> ✓ Run Checkstyle		10s
> ✓ Run SpotBugs		18s
> ✓ Generate Coverage Report		2s
> ✓ Upload Coverage Report		0s
> ✓ Post Set up JDK 21		3s
> ✓ Post Run actions/checkout@v4		0s
> ✓ Complete job		0s

Pipeline Benefits and Issue Detection

The CI pipeline identifies the following types of issues:

- **Compilation failures:** Syntax errors, missing imports, or incompatible API changes are caught immediately. Example: A refactor that renamed GeometryService methods would fail until all callers are updated.
- **Test regressions:** Any commit that breaks existing functionality is detected. Example: Modifying the A* algorithm that causes DroneRoutingServiceTest to fail would block the merge.
- **Environment inconsistencies:** Code that works locally but fails in CI reveals hidden dependencies. Example: Tests relying on local file paths or environment variables would fail in the clean CI environment.
- **Coverage degradation:** JaCoCo coverage reports track whether new code maintains coverage thresholds. A future enhancement would enforce minimum 90%-line coverage as a build gate.

Future Improvements

- **End-to-end testing:** Implement Selenium or Playwright tests to simulate user interaction with any future frontend, ensuring users can access REST endpoints successfully.
- **Automated deployment:** Extend pipeline to deploy to AWS ECS or Kubernetes on successful main branch builds, implementing true CD.
- **Performance regression testing:** Add pipeline stage that runs benchmark tests and fails if response times exceed thresholds (e.g., >5 seconds for pathfinding).
- **Security scanning:** Integrate OWASP Dependency-Check to identify vulnerabilities in third-party libraries, critical for a medical delivery system.

Conclusion

The combination of manual review techniques (code walkthroughs, checklists, static analysis) and automated CI/CD provide comprehensive quality assurance for MedSupplyDrones. Reviews identified issues that testing alone missed – particularly around floating-point precision and edge cases in the pathfinding algorithm. The CI pipeline ensures these fixes are not regressed by running all tests on every commit. While the current implementation covers Source, Build, and Test stages, extending to automated deployment would complete the DevOps workflow for production readiness.